# 6. IMPLEMENTING THE SIMULATED ANNEALING ALGORITHM

```
import random
import math

def objective(x):
    return (x - 3) ** 2



def simulated_annealing(initial_state, initial_temp,
                        cooling_rate, min_temp, max_iter, seed):

    random.seed(seed)

    current_state = initial_state
    current_cost = objective(current_state)

    best_state = current_state
    best_cost = current_cost

    temperature = initial_temp

    for iteration in range(max_iter):

        if temperature <= min_temp:
            break

        candidate_state = current_state + random.uniform(-1, 1)
```

```
        candidate_cost = objective(candidate_state)

        delta = candidate_cost - current_cost

        if delta < 0:
            accepted = True
        else:
            probability = math.exp(-delta / temperature)
            accepted = random.random() < probability

        if accepted:
            current_state = candidate_state
            current_cost = candidate_cost

        if current_cost < best_cost:
            best_state = current_state
            best_cost = current_cost

        temperature *= cooling_rate

    return best_state, best_cost


# Input
initial_state = float(input("Initial State: "))
initial_temp = float(input("Initial Temperature: "))
cooling_rate = float(input("Cooling Rate: "))
min_temp = float(input("Minimum Temperature: "))
max_iter = int(input("Maximum Iterations: "))
```

```
seed = int(input("Random Seed: "))


# Execution
best_state, best_cost = simulated_annealing(
    initial_state, initial_temp, cooling_rate,
    min_temp, max_iter, seed
)


# Output
print("\nBest State (x):", f"{best_state:.6f}")
print("Minimum Function Value:", f"{best_cost:.8f}")
print("The result should be a low-cost value near the global minimum x = 3.")
```

**Output:**

Test Case 1: Initial State at the Optimum

```
======= RESTART: C:/Users/Admin/Documents/AI lab/7. simulated anneling.py
======
Initial State: 3
Initial Temperature: 100
Cooling Rate: 0.95
Minimum Temperature: 0.01
Maximum Iterations: 500
Random Seed: 1

Best State (x): 3.000000
Minimum Function Value: 0.00000000
The result should be a low-cost value near the global minimum x = 3.
```

Test Case 2: Initial State Below the Optimum

```
======= RESTART: C:/Users/Admin/Documents/AI lab/7. simulated anneling.py
======
Initial State: -5
Initial Temperature: 100
Cooling Rate: 0.95
Minimum Temperature: 0.01
Maximum Iterations: 500
Random Seed: 42

Best State (x): 2.997881
Minimum Function Value: 0.00000449
The result should be a low-cost value near the global minimum x = 3.
```

Test Case 3: Initial State Above the Optimum

```
======= RESTART: C:/Users/Admin/Documents/AI lab/7. simulated anneling.py
======
Initial State: 9
Initial Temperature: 100
Cooling Rate: 0.95
Minimum Temperature: 0.01
Maximum Iterations: 500
Random Seed: 7

Best State (x): 2.993599
Minimum Function Value: 0.00004097
The result should be a low-cost value near the global minimum x = 3.
```

Test Case 4: Low Initial Temperature

```
======= RESTART: C:/Users/Admin/Documents/AI lab/7. simulated anneling.py
======
Initial State: 0
Initial Temperature: 1
Cooling Rate: 0.9
Minimum Temperature: 0.01
Maximum Iterations: 500
Random Seed: 10

Best State (x): 2.995676
Minimum Function Value: 0.00001870
The result should be a low-cost value near the global minimum x = 3.
```

Test Case 5: Fast Cooling Schedule

```
======= RESTART: C:/Users/Admin/Documents/AI lab/7. simulated anneling.py
======
Initial State: 10
Initial Temperature: 100
Cooling Rate: 0.5
Minimum Temperature: 0.01
Maximum Iterations: 100
Random Seed: 21

Best State (x): 5.895203
Minimum Function Value: 8.38220061
The result should be a low-cost value near the global minimum x = 3.
```